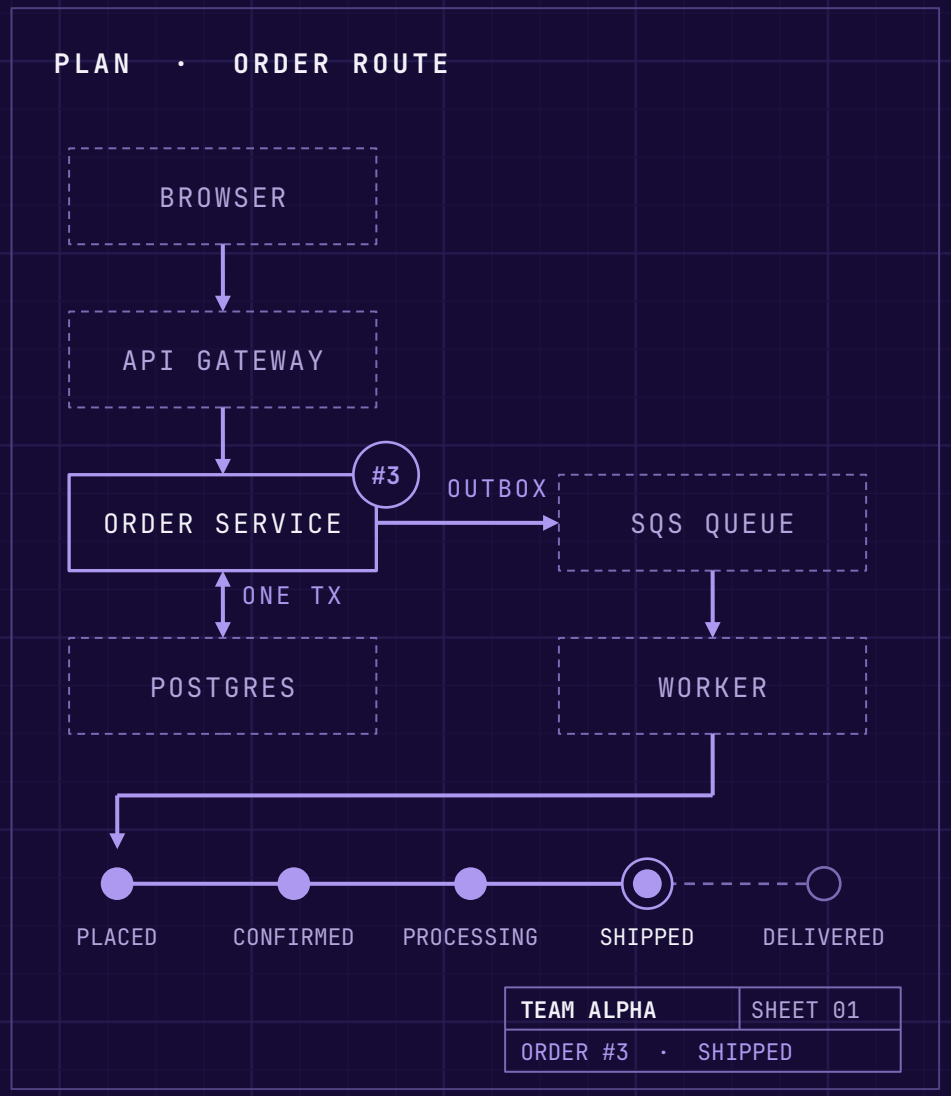
KIT ROOM · FINAL DEMO

One order, followed all the way down.

How Team Alpha took a nine-service order platform from provided source code to Amazon EKS in four weeks, on a $200 budget, and built it for the orders that go wrong.



TEAM ALPHA · CODERCO PRODUCTION ENGINEERING SIMULATION · 13 OCTOBER 2026

Kit Room equips new starters from one page

An operations team picks the parts a new starter needs. Kit Room prices them from live stock, takes payment, ships them and keeps everyone told.

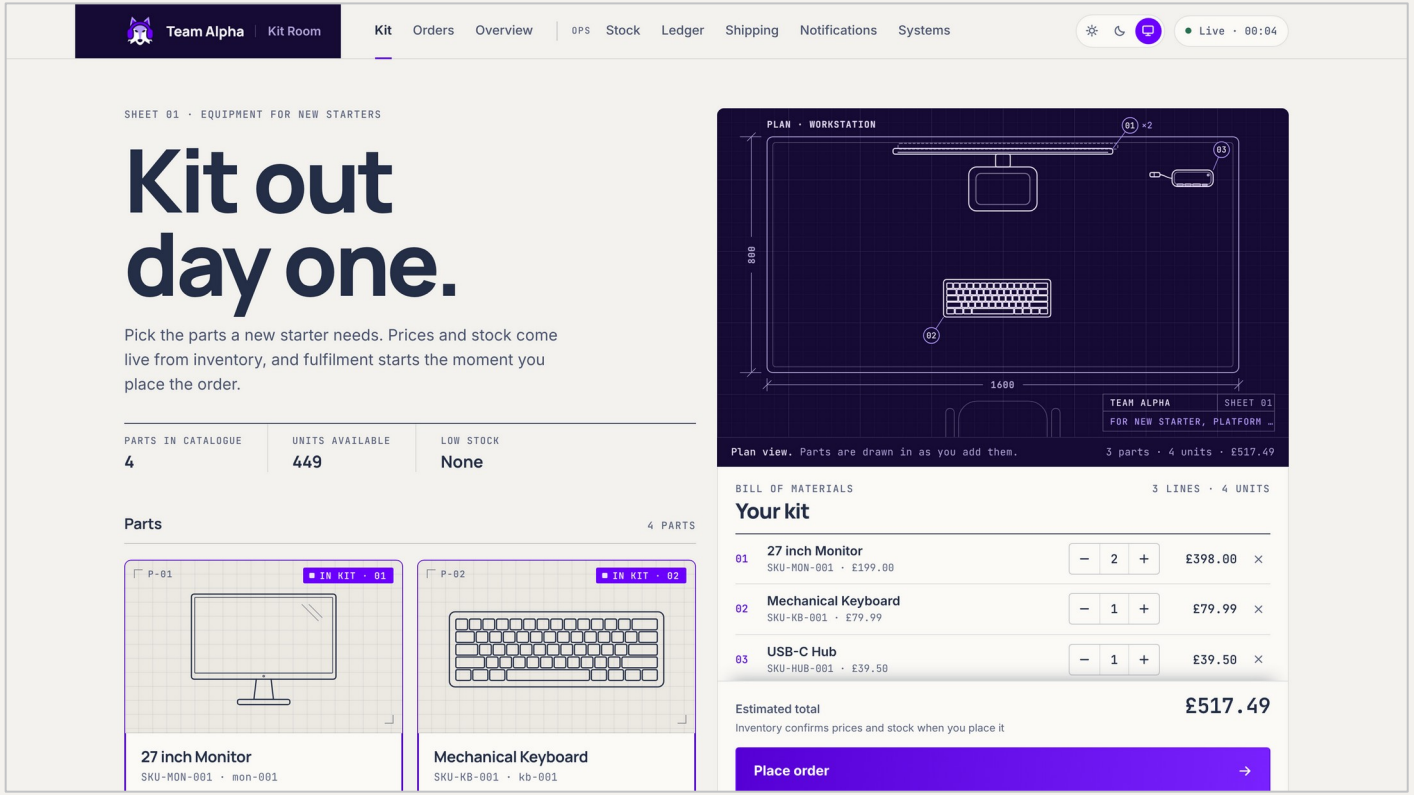
WHO IT SERVES

The person ordering
builds a kit, places the order, tracks it.

Operators
manage stock, payments and shipping.

ONE JOURNEY

01 Build a kit
02 Place the order
03 Fulfilment runs by itself
04 Track it to delivery



THE KIT PAGE · CURRENT BUILD, RUN LOCALLY, 13 OCT

Out of the box, no order got past pending

AS PROVIDED · ORDER-SERVICE/MAIN.GO

```
data, _ := json.Marshal(event)
log.Printf("Event → SQS: %s", string(data))
// Students implement actual SQS SendMessage here
```

AS PROVIDED · WORKER/MAIN.GO

```
func receiveSQSMessages(ctx context.Context,
        queueURL string) []string {
    // Students implement with AWS SDK SQS ReceiveMessage
    return nil
}
```

Publishing was a log line. Nothing read the queue. No tests.

168 test functions in 20 files. The provided services had none.

BUILT INTO THE APP

Transactional outbox and SQS publisher

The worker: reserve, charge, confirm, commit

Idempotency keys on orders and payments

A route allow-list at the gateway

BUILT AROUND IT

9 multi-stage Dockerfiles, distroless, non-root

Terraform: 6 modules, 3 environments, a bootstrap

One Helm chart for 9 services, ArgoCD app-of-apps

8 GitHub Actions workflows

Five aims, shaped by a $200 budget

CONSTRAINTS

$200

AWS spend CoderCo reimburses per team. Every cost needs a reason.

STAKEHOLDER PRIORITY ORDER, 14 SEP

1 Cost	2 Security	3 Uptime	4 Features

FIXED BY THE BRIEF

EKS across 3 AZs · Karpenter · Traefik behind an NLB · cert-manager and ExternalDNS · ArgoCD app-of-apps · Postgres and Redis on StatefulSets · GitHub OIDC · image scanning

4 weeks · 5 engineers · 1 AWS account

WHAT WE SET OUT TO DO

01 Never lose an accepted order
even when a downstream service or AWS is briefly down.

02 Never charge twice
a retry, a redelivery or a double click changes nothing.

03 Hold no long-lived AWS keys
nothing in Git, CI or an image can reach AWS on its own.

04 Change the platform only through Git
Terraform for AWS, ArgoCD for the cluster.

05 Ship one service without touching eight
and roll it back the same way.

AIMS, NOT GUARANTEES. SLIDE 18 SAYS WHICH ARE PROVEN.

Watch one order travel

01

Build a kit

NOTICE

Prices and stock come live from inventory. The browser never sets a price.

02

Place the order

NOTICE

"Placed" comes back at once: the order is saved and queued before any work begins.

03

Wait a few seconds

NOTICE

It turns "Confirmed" by itself: a background worker reserved the stock and took the payment.

04

Ship it

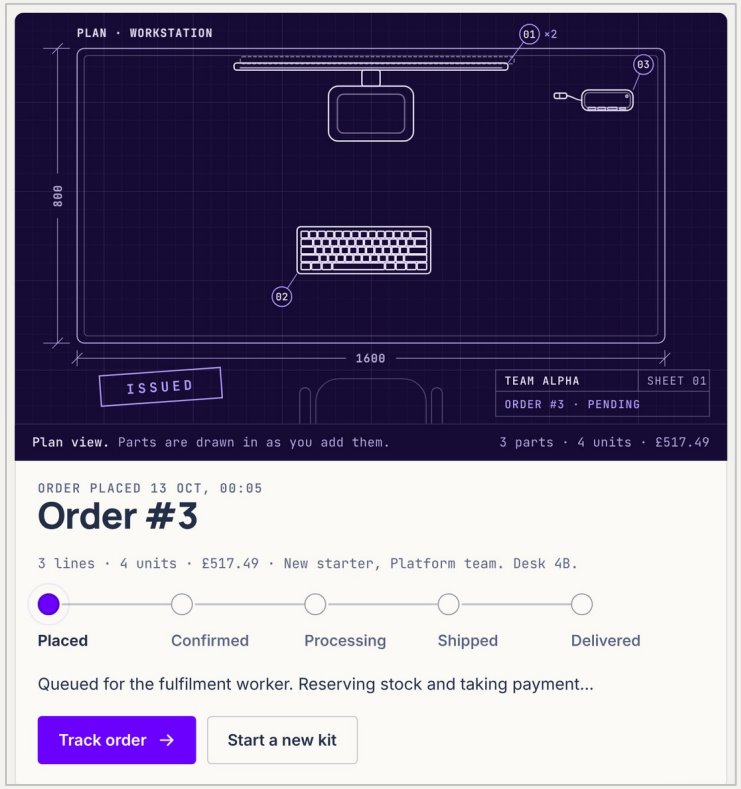
NOTICE

Shipment, tracking number and emails appear, and each is created once.

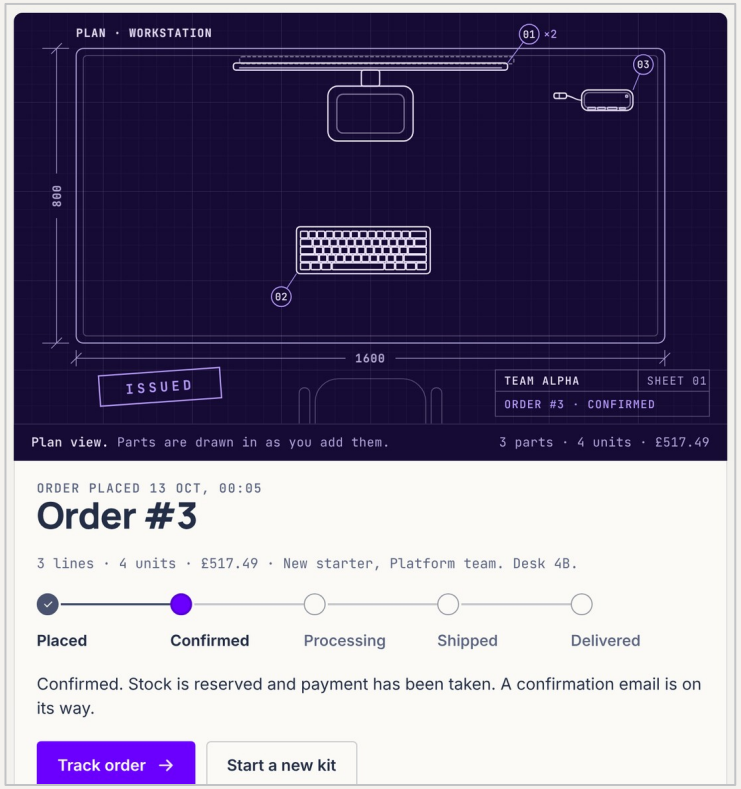
LIVE dashboard.dev.eksv2.naimaosmann.com **IF DEV IS DOWN** slides 6 and 7 show the same journey from this build

Answered at once, confirmed seconds later

1 MOMENT OF THE CLICK



2 SECONDS LATER, NO CLICK



1 **Placed is the API's 201 reply.**

The order, its history row and its event were saved together. Nothing has been processed yet.

2 **Nobody pressed confirm.**

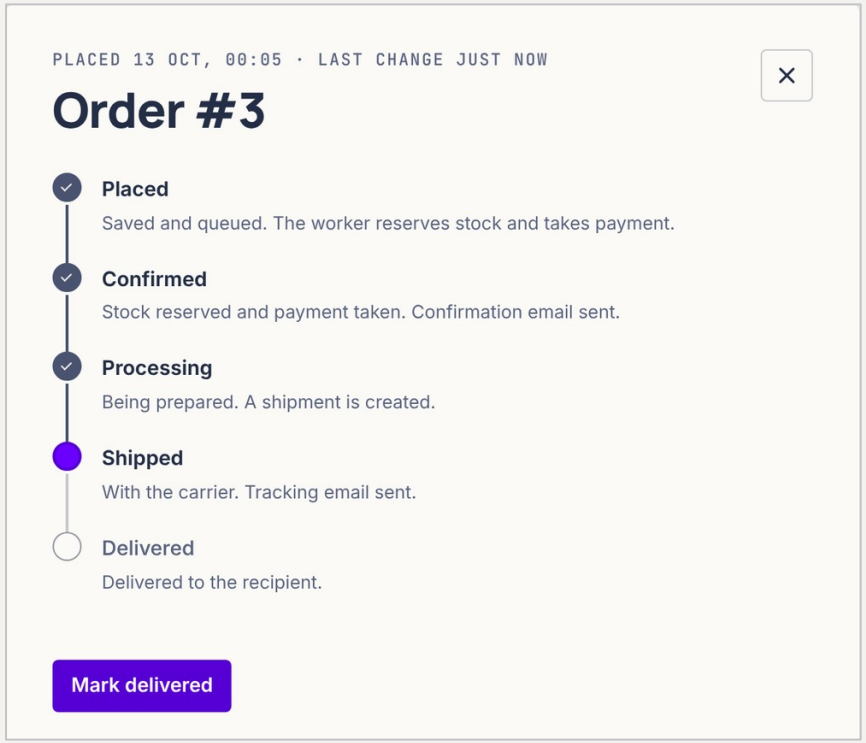
The worker read the event from SQS, reserved stock, charged the card and confirmed the order.

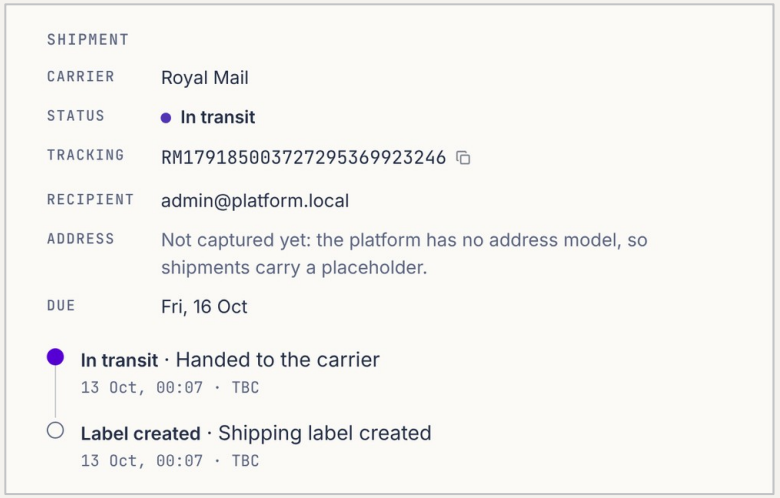
● **Stock leaves the shelf last.**

It is committed only after the charge succeeds, so a declined card never takes stock.

ORDER #3, CURRENT BUILD RUN LOCALLY ON 13 OCT. A LIVE ORDER WILL HAVE ANOTHER NUMBER.

Every later step is visible, and happens once







Shipping follows the status.

Processing creates the shipment; shipped moves it in transit and emails the tracking number.

Each email is sent once.

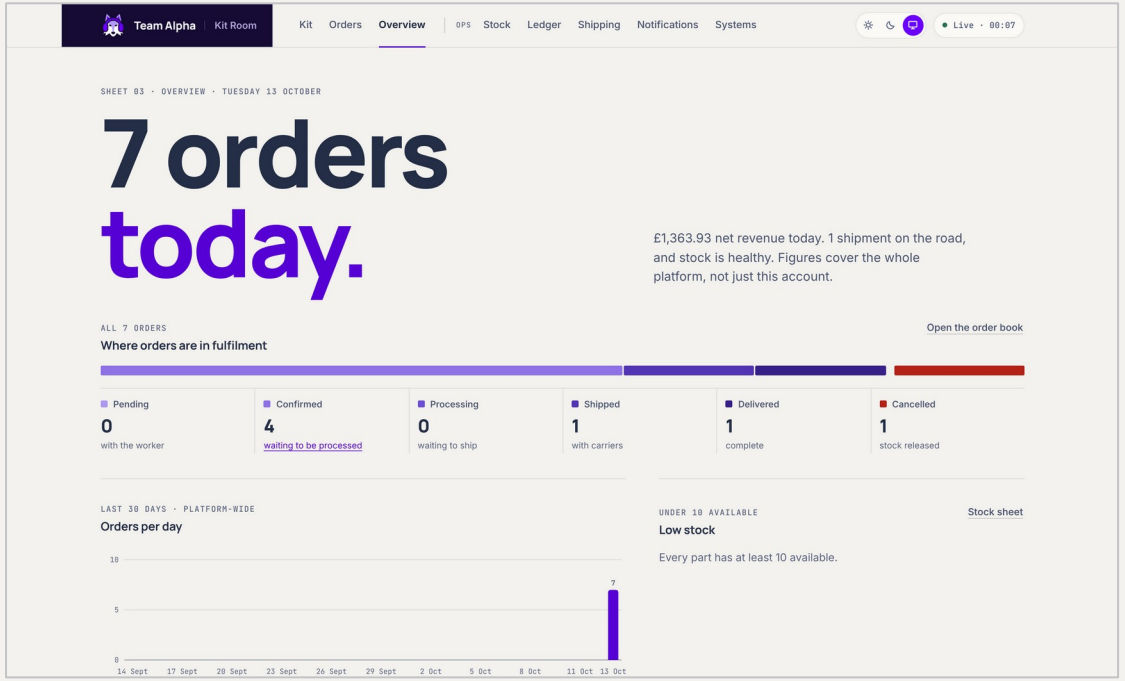
It carries a dedupe key, template plus order, so a replayed event sends nothing new.

A gap we show, not hide.

There is no address model yet, and the page says so.

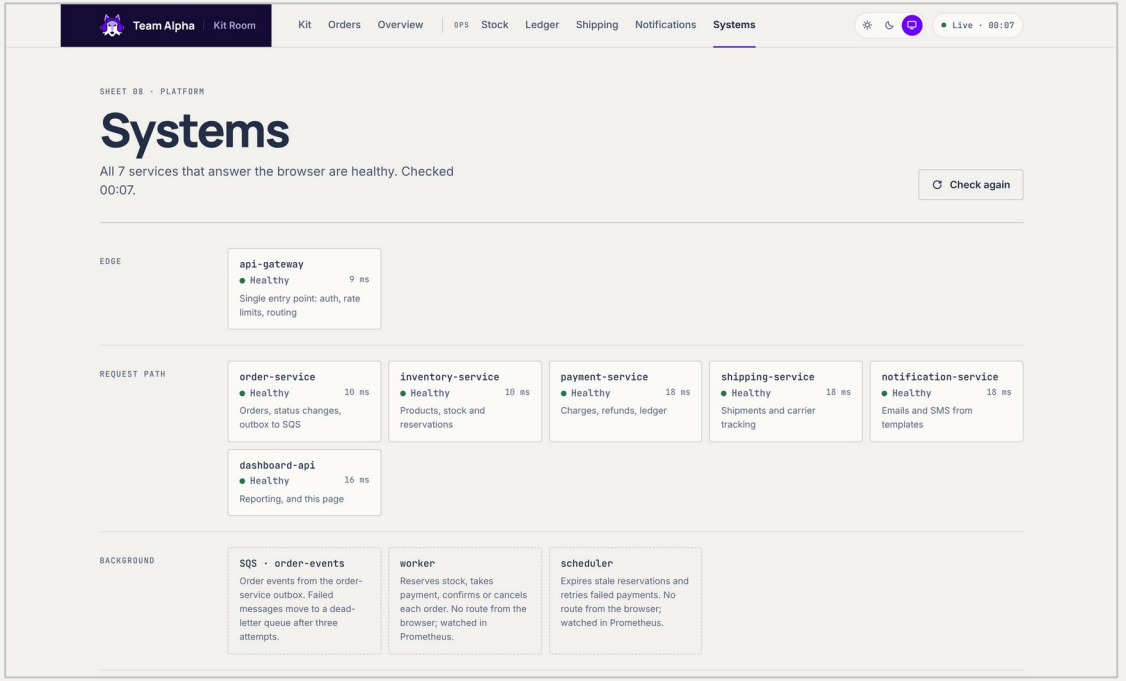
ORDER #3 MOVED TO PROCESSING AND SHIPPED FROM THE ORDERS PAGE.

The same page runs the platform



OVERVIEW

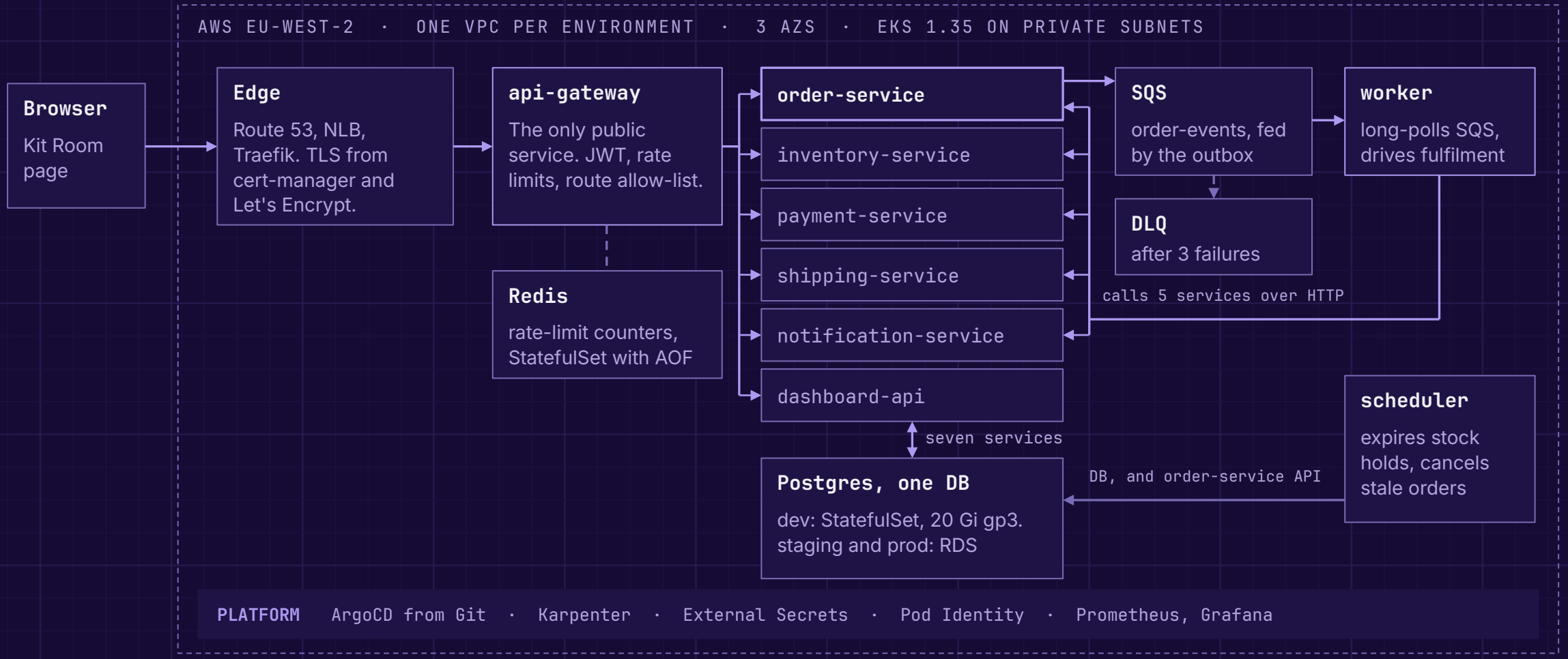
Every figure comes from a service API through the gateway. Nothing is invented on the page.



SYSTEMS

Seven services answer the browser. The worker and scheduler have no route from outside, by design.

One entry point, nine services, one queue



Accepting an order: all or nothing

1 Browser

POST /api/orders with an Idempotency-Key; a retry reuses the same key.

→

2 api-gateway

Valid JWT? Under the rate limit? A route the outside may call?

→

3 order-service

Seen this key? Replay the first answer. Otherwise price every line from inventory.

→

ONE POSTGRES TRANSACTION

`INSERT` **idempotency_keys**
claim (customer, key): the guard

`INSERT` **orders**
order #3, status pending

`INSERT` **order_events**
order_created, for history

`INSERT` **order_outbox**
order.created, waiting to send

COMMIT. All four rows exist, or none do.

→

4 Reply

201 Created. Order #3, pending. The customer has an order number.

THEN, EVERY SECOND

The outbox relay claims unsent rows (FOR UPDATE SKIP LOCKED, so replicas never collide), sends each to SQS and marks it sent.

IF SQS IS UNREACHABLE

The row stays unsent, attempts + 1, and is retried next tick. New orders are still accepted.

THE PRICE WE PAY

A crash after SQS accepts but before "sent" is saved sends the event twice, so consumers must be idempotent.

Processing: a worker that can safely run twice

SQS order-events

Long poll 20 s, up to 10 messages. A message is hidden for 60 s while it is worked on.

third failure

order-events-dlq

The third failed receive lands here. Kept 14 days. Redrive once the cause is fixed.

WORKER, FOR ORDER #3

0 **Check the order first**

pending: run below. confirmed: only commit stock. cancelled: undo.

1 **Reserve stock**

inventory /reserve. A repeat returns the same hold.

2 **Charge**

payment /charge, key charge-order-3. A repeat replays the first answer.

3 **Confirm**

order /status, compare-and-set: only if still pending.

4 **Commit stock, delete message**

Stock leaves the shelf only now.

THREE POSSIBLE ENDINGS

Success

Message deleted. Status events follow: confirmation email, then shipment and tracking as the order moves on.

A final "no"

Out of stock or card declined: release stock, cancel, tell the customer, delete. Retrying would not change the answer.

An error or timeout

Not deleted. SQS hands it out again after 60 s. On the third failure it moves to the DLQ.

We broke it first, then made retries safe

WHERE A REPEAT IS ABSORBED

LAYER	A REPEAT LOOKS LIKE	WHAT ABSORBS IT
Request	double click, client retry	Idempotency-Key. Primary key on (customer, key) lets one request win.
Worker	SQS redelivery	Check the order first. Status changes are compare-and-set.
Payment	charge retried	Key charge-order-N, and a unique index: one completed charge per order.
Email	event replayed	Dedupe key template:order, held by a unique index.

168 test functions pass with the race detector on, re-run against Postgres, Redis and a local SQS on 13 Oct.

REPRODUCED BEFORE THE FIX

25 of 30 — NOW 0

runs where a concurrent confirm and cancel both succeeded

−57 — NOW 0

stock "reserved" after 20 concurrent releases of a hold of 3

6 rows — NOW 1

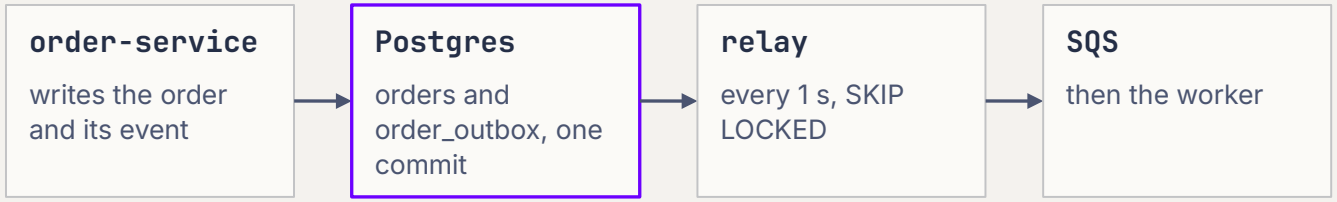
in payments after 6 retries of one charge

Team fix-evidence log, 23 to 24 Sep, against a Postgres 16 test database.

SQS behind a transactional outbox

THE PROBLEM

Saving an order (Postgres) and announcing it (SQS) are two systems. If the announcement is lost after the save, nothing ever reserves stock or takes payment.



The order and its event share one commit. If SQS or the NAT path is down, the event row waits and the relay retries every second. Lost becomes late.

ALTERNATIVES WE WEIGHED

Kafka on Strimzi: three brokers on volumes, producer rewrites. Publish after commit: a crash loses the event. SQS FIFO: ordering we do not need.

WE GAINED

- SQS outage: orders still accepted, events wait
- Retries, visibility timeout and DLQ built in
- Managed and near free at our volume
- The services were already written for SQS

WE GAVE UP

- At-least-once: duplicates are possible, so every consumer step must be safe to repeat
- No ordering, no replay once a message is deleted
- Up to about 1 s extra before processing starts
- Payment and shipment events are not on an outbox yet

EVIDENCE outbox_test.go (relay retry, shutdown mid-batch); local drill 24 Sep: payment stopped, 3 receives, DLQ, redrive, order processed.

Postgres in the cluster for dev, RDS beyond

THE PROBLEM

An order, its stock hold, its payment and its outbox row commit together, so one relational database. The brief wants a StatefulSet; production wants failover and backups we would rather not hand-run.

DEV

StatefulSet postgres-0

20 Gi encrypted gp3, daily EBS snapshot, 7 kept

RAN ON EKS

STAGING

RDS db.t4g.medium

single AZ, smaller and cheaper

IN CODE ONLY

PROD

RDS db.m7g.large

Multi-AZ. Terraform refuses prod without deletion protection and 7+ days of backups

IN CODE ONLY

ALTERNATIVES WE WEIGHED

StatefulSet everywhere: we own failover and backups. RDS everywhere: skips the StatefulSet the brief grades, costs while idle. CloudNativePG: one more operator.

WE GAINED

- Dev meets the brief and costs only a volume
- Prod gets managed failover and point-in-time restore
- Services never know: External Secrets builds DATABASE_URL

WE GAVE UP

- Dev Postgres is one pod on one AZ's disk: lose the AZ, writes stop until a restore
- Two database setups to understand
- RDS has never run live: staging and prod are not deployed

EVIDENCE dev StatefulSet and SnapScheduler (daily, keep 7) in Git and on EKS; snapshot class confirmed on dev 10 Oct. A restore with a data check is not recorded.

CI builds, Git decides, ArgoCD deploys

THE PROBLEM

Nine services ship on their own schedules. A change to one must not redeploy or risk the other eight, and the pipeline should hold no keys to the cluster.



Prod never auto-syncs: promotion copies a tested tag dev to staging to prod by pull request, and someone syncs prod in ArgoCD.

ALTERNATIVES WE WEIGHED

CI runs helm upgrade: cluster keys in CI. Flux: equally valid, the team knew ArgoCD. Kustomize overlays: simpler patches, logic copied nine times.

WE GAINED

- Rollback is one revert, for one service
- CI has no cluster credentials at all
- ArgoCD reverts hand edits (self-heal)
- Only changed services are built

WE GAVE UP

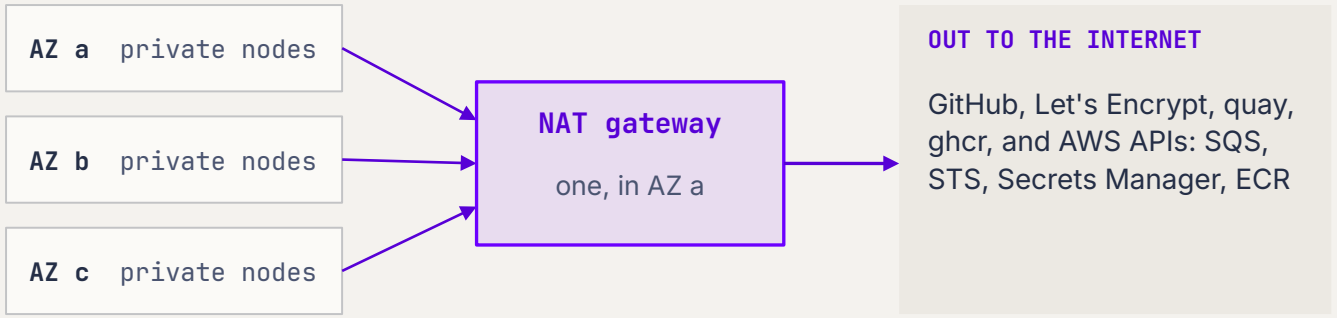
- A deploy waits for ArgoCD to notice the commit
- A bot must be allowed to write to main
- One chart bug reaches all nine (schema and kubeconform guard it)
- Canary analysis passes when there is no traffic

EVIDENCE smoke test on EKS dev, 8 Oct: 8 failures before the hardening deploy, all steps passing after. Zero downtime is not proven (slide 18).

One NAT gateway, on purpose

THE PROBLEM

Private nodes must reach the internet: GitHub for ArgoCD, Let's Encrypt for certificates, public image registries, and AWS APIs for the pods. The brief says avoid NAT if you can.



S3 GATEWAY ENDPOINT free: image layers in S3 skip the NAT

ALTERNATIVES WE WEIGHED

A NAT per AZ: survives an AZ loss at 3× the hourly cost. Interface endpoints: 6+ services × 3 AZs = 18 billed endpoints, and still no route to GitHub.

WE GAINED

- The cheapest design that still reaches GitHub and Let's Encrypt
- S3 traffic, including image layers, skips the NAT through a free gateway endpoint

WE GAVE UP

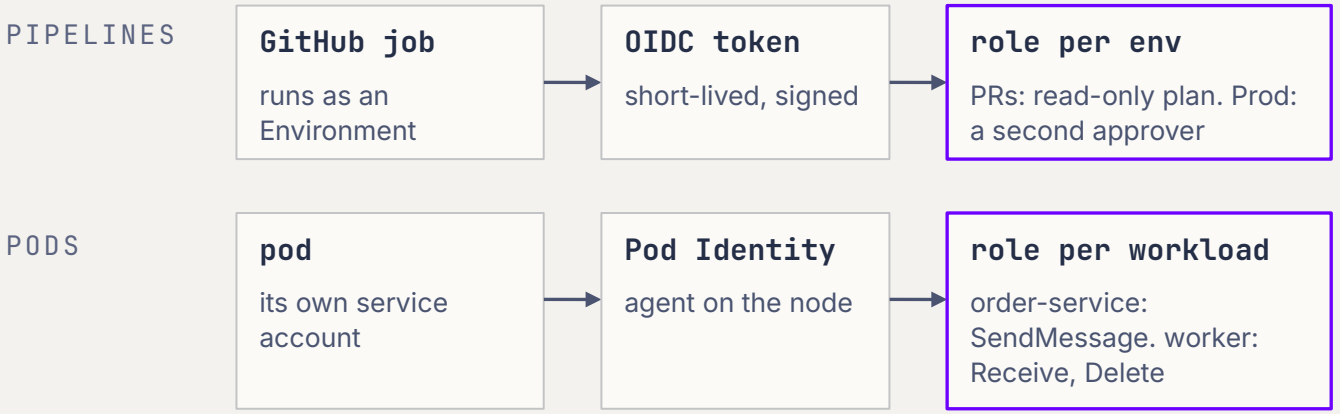
- One AZ outage cuts outbound traffic for all three AZs
- Image pulls, certificate renewals, ArgoCD syncs and SQS calls fail until it returns
- Traffic inside the cluster carries on

EVIDENCE modules/vpc: one aws_nat_gateway, one S3 gateway endpoint. The outbox keeps orders safe while SQS is unreachable.

No long-lived AWS keys anywhere

THE PROBLEM

Pipelines and pods both call AWS. A static access key in a secret or an image works anywhere, for anyone who copies it, until someone rotates it.



ALTERNATIVES WE WEIGHED

IRSA: an OIDC provider per cluster, named in every trust policy. Static keys in GitHub secrets: simple, and exactly what the brief rules out.

WE GAINED

- Nothing in Git, CI or an image can reach AWS on its own
- Each workload can do one job: order-service sends, the worker receives and deletes
- The same role definitions work on every cluster

WE GAVE UP

- We departed from the brief's wording (IRSA) while keeping its intent
- Needs the Pod Identity agent on every node, and no Fargate
- More roles to reason about

EVIDENCE bootstrap/github-actions-roles.tf (trust pinned to org, repo and Environment); modules/pod-identity; IMDSv2 hop limit 1, so pods cannot borrow the node role.

What we have proven, and what we have not

DEMONSTRATED ON EKS DEV

- All nine services healthy behind HTTPS at a real DNS name (Let's Encrypt staging certificate)
- The full order lifecycle, including cancel and refund: smoke test, 8 Oct, every step passing
- Internal routes refused at the gateway (404 from outside)
- Built from Git: Terraform pipelines, then ArgoCD bootstrapping the platform in sync waves
- Daily EBS snapshots of the dev database scheduled

PROVEN IN TESTS, NOT UNDER LOAD

- 168 test functions, all passing with the race detector on (re-run 13 Oct)
- Duplicate and concurrent requests: one order, one charge, one email
- Payment outage: retries, DLQ, redrive, order processed (local drill)
- The whole journey on the current build, run locally on 13 Oct

DEFINED IN CODE, NOT YET RUN

- Staging and prod, including RDS and promotion between them
- k6 load tests: plan and scripts written, never executed
- A restore checked against known data; a chaos drill
- Argo Rollouts canary against a deliberately bad release

Not claimed: zero downtime. A 30 to 60 second blip during an ArgoCD sync on 10 Oct still has no root cause.

Limits that matter, and what we would do next

LIMITS THAT MATTER

Anyone can sign in.

Login accepts any email, and every signed-in user sees every order.

Connections run out first.

Seven pools allow 115 connections against Postgres's default 100.

Two event types bypass the outbox.

Payment and shipment events are only logged, and the DLQ alarm notifies nobody.

Dev Postgres lives in one AZ.

Lose that AZ and writes stop until a snapshot is restored elsewhere.

NEXT, IN THIS ORDER

01 **Measure before tuning.**

Run the k6 plan on dev, then size the pools or add PgBouncer from what it shows.

02 **Real identity.**

Customer accounts and ownership checks at the gateway, before any real user.

03 **Rehearse the bad day.**

Stand up staging, restore a snapshot against known data, roll back a bad release on purpose.

Engineered for the order that goes wrong.

IF SQS GOES QUIET

orders are still accepted. Their events wait in the outbox.

IF A REQUEST REPEATS

nothing doubles. Keys and state checks absorb it.

IF THE REPO LEAKS

it holds no AWS key. Pipelines and pods borrow short-lived roles.

Built for a $200 budget, by five engineers, in four weeks.

A change to payment-service, from commit to traffic

1 Pull request

Only payment-service changed, so only its job runs: go vet, go test with the race detector, docker build, Trivy (fails on HIGH or CRITICAL).

2 Infra checks, if touched

helm lint and kubeconform on every rendered service; terraform fmt, validate, Checkov and a plan per environment.

3 Merge to main

The image is built once, tagged with the service and commit, and pushed to ECR. No other service is rebuilt.

4 Tag commit

A GitHub App commits the new tag to the dev values file for payment-service. Git now says what dev should run.

5 ArgoCD sync

Only payment-service-dev changes. The other eight Applications see no diff.

6 Canary

Argo Rollouts sends 50%, checks error rate (5% max) and p95 (500 ms max) in Prometheus, then 100%.

7 Promote

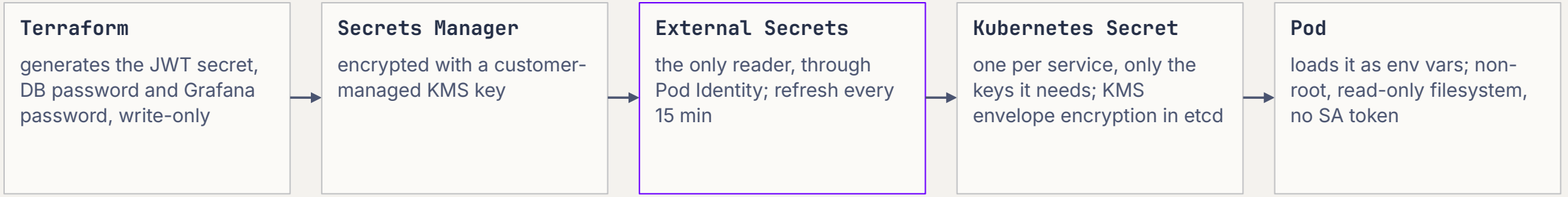
The Promote workflow opens a PR copying the tested tag to staging, then prod. Prod needs approval and a manual sync.

8 Roll back

Revert the tag commit. The old image is still in ECR, so it is one rollout.

The canary landed last, in the 13 Oct sync. Not yet tried against a bad release; with no traffic its checks pass by default.

From Secrets Manager to a pod, and what rotation does



ROTATED	WHO RESTARTS	THE GAP
JWT secret	api-gateway only (Reloader watches its Secret)	Tokens signed with the old secret stop working; users sign in again.
Database password	The seven services that use Postgres, one pod at a time	Until the refresh lands (up to 15 min), new connections from old pods fail; open ones keep working.
Who never restarts	The worker: it holds no secret at all	It reaches SQS through Pod Identity, not a key.

What scales, on what signal, and what breaks first

COMPONENT	SCALES ON	LIMITS IN DEV
api-gateway	HPA, CPU 70%	2 to 3 pods
Six HTTP services	HPA, CPU 70%	1 to 3 pods each
worker	fixed	1 pod. Queue depth (KEDA) is the better signal
scheduler	never	1 pod. Chart refuses more, uses Recreate
Postgres, Redis	never	1 pod each, StatefulSet
Nodes	Karpenter, pending pods	3 base t3.medium; on-demand only, 16 vCPU and 64 GiB cap

WHAT BREAKS FIRST

Reasoned from configuration, not measured.

Connections. 115 allowed by the pools, 100 by Postgres. Each new pod adds up to 25.

Postgres itself. One writer on one volume.

Redis down. The gateway fails open: up, but unthrottled.

Worker behind. SQS keeps the backlog for 4 days. Slower, not lost.

What happens when things fail, and how we know

SCENARIO	WHAT HAPPENS	HOW WE KNOW
Worker fails 3 times	Not deleted, back after 60 s; third failure goes to the DLQ (kept 14 days). Redrive after the fix; steps are safe to repeat.	Local drill, 24 Sep
payment-service down 2 min	Orders still accepted. Charges fail and the messages return; most succeed once it is back. Stock is held 15 min.	Code path and local drill
SQS or the NAT unreachable	Orders accepted; outbox rows wait and the relay retries every second. The worker pauses.	outbox tests
Same order sent twice	The same Idempotency-Key returns the first order.	Smoke test on EKS, 8 Oct
Bad notification-service image	New pods fail readiness; old ones keep serving. Revert one tag commit; eight services untouched.	Design; not drilled
Postgres pod deleted	StatefulSet recreates it on the same volume; services reconnect.	Expected; not drilled
AZ holding the dev DB lost	Pod stays Pending. Restore the latest snapshot in another AZ, losing writes since.	Procedure only

Dev database: storage, snapshots and restore

SETTING	VALUE
Volume	20 Gi gp3, encrypted (StorageClass)
Placement	WaitForFirstConsumer: disk in the pod's AZ
Snapshots	SnapScheduler, daily 01:00 UTC, keep 7
Snapshot class	ebs-csi, deletionPolicy Delete
Redis	AOF on a 1 Gi gp3 volume (brief: 10 Gi)
Migrations	CREATE TABLE IF NOT EXISTS on start-up

RESTORE: A NEW DISK, NEVER THE OLD ONE

1. Take a snapshot of data-postgres-0; wait for READY
2. Scale the Postgres StatefulSet to 0
3. Delete the PVC data-postgres-0
4. Create it again with the snapshot as dataSource
5. Scale back to 1, then check known orders are back

Status. The snapshot class and schedule run on dev. A restore checked against known data is not in our records. Schema changes: the plan is golang-migrate as an ArgoCD PreSync Job, with changes that expand first and contract later. Not built yet.

Three environments in code, one that has run

	DEV	STAGING	PROD
Status	**The one that has run; torn down between sessions to save cost**	**Defined in code, never deployed**	**Defined in code, never deployed**
VPC	10.6.0.0/16	10.7.0.0/16	10.8.0.0/16
Database	Postgres StatefulSet	RDS db.t4g.medium, single AZ	RDS db.m7g.large, Multi-AZ
Nodes (base)	3 x t3.medium, max 4	3 x t3.medium, max 5	3 x t3.medium, max 6
ArgoCD sync	Automatic	Automatic	Manual
Terraform apply	Saved plan, manual run	Plan made on main only	Plan on main, plus a second approver
Separation	Each has its own VPC, cluster, Terraform state, GitHub Environment and IAM role, in one AWS account		

One account, not three: cross-account access for the shared ECR, Route 53 zone and state did not fit the time we had.

Known gaps we would name before you do

GAP	WHY IT MATTERS	FIRST FIX
Open login	Any email and password gets a token; every user sees every order.	Accounts, ownership checks
One shared database user	A bug in one service can read or change another's tables.	A role per service
Redis	No password, no NetworkPolicy, no TLS.	AUTH and a NetworkPolicy
Carrier webhook	Public and rate limited, but unsigned.	HMAC signature
Browser headers	No CSP or HSTS yet.	Set them at Traefik
Money as float	Totals are rounded to pence, but summed as floats in Go.	Integer pence
Dev certificate	Let's Encrypt staging: browsers warn.	Production issuer

Where the money goes, and what we did about it

Tear down when idle

Environments are rebuilt from Git, so dev did not run around the clock.

One NAT gateway

Not one per AZ, and no paid interface endpoints (slide 16).

Capped autoscaling

Karpenter on-demand only, 16 vCPU and 64 GiB at most, consolidation after 5 min.

In-cluster data in dev

Postgres and Redis share nodes we already pay for; only volumes cost extra.

Small fixed footprint

One shared ECR repository keeping the newest 40 images; control-plane logs kept 30 days.

SPEND ON RECORD

about $50

of AWS spend recorded by 10 Oct, mostly EC2, against the $200 cap.

Billed by the hour whenever dev is up: the EKS control plane, three base nodes, the NAT gateway and the NLB. That is why tear-down is the biggest lever.